FlowGuard: Contract-Aware Runtime Verification, Bounded Two-Tier Recovery & 4-State Transactional Idempotency for Agentic Workflows

Author: Iyed Mohamed (Final-Year Computer Engineering Student, CTI / EUR-ACE Accredited Master's Equivalent, ESPRIT, Tunis, Tunisia)

ORCID: `0009-0008-9493-7510` | **Repository:** `https://github.com/iyedM/flowguard` (Release `v0.2.0`) | **Companion Paper:** *CEGAR-Harness* (`DOI: 10.5281/zenodo.23731911`)

Abstract — Workflow automation platforms such as **n8n**, **LangGraph**, and **Model Context Protocol (MCP)** tool servers increasingly delegate operational decisions in **AIOps incident remediation**, **DevSecOps CVE/secret triage**, and **Cloud/GPU FinOps provisioning** to Large Language Models (LLMs). While workflow engines reliably route JSON payloads between nodes, they provide no formal runtime verification at the boundary where stochastic LLM outputs trigger external state-mutating APIs. Consequently, production agentic workflows suffer from four interacting failure modes: (1) schema drift and markdown-wrapped JSON truncation, (2) silent cross-field semantic invariant violations, (3) duplicate side-effect mutations caused by blind retries after post-commit HTTP timeouts, and (4) indirect prompt-injection attacks embedded in untrusted logs, pull requests, or tickets. We present **FlowGuard**, a neuro-symbolic runtime verification, bounded recovery, and 4-state transactional idempotency middleware. Across a **720-run controlled multi-domain benchmark** (180 scenarios × 4 strategies), FlowGuard improves the State-Verified Success Rate (SVSR) from **18.89%** [95% Wilson CI: 13.84%, 25.23%] (native n8n) and **27.22%** (naive retry) to **91.67%** [86.71%, 94.89%], while driving the Unsafe Side-Effect Rate from **67.78%** down to **0.00%**. A **900-run 5-way ablation study** isolates the exact contribution of each mechanism, and a **30-run live local GPU evaluation** with `llama3:latest` (8B Q4_0 via Ollama on an NVIDIA RTX 2050) achieves **93.33% SVSR** [70.18%, 98.81%] and **0.00% unsafe side-effects** (vs. 20.00% SVSR and 73.33% unsafe side-effects under naive retry).

1. INTRODUCTION, RESEARCH QUESTION & EPISTEMIC SCOPE

Modern enterprise automation increasingly replaces static conditional branches with LLM-powered reasoning nodes inside orchestration graphs (e.g., n8n AI Agent nodes, LangGraph `StateGraph` nodes, and MCP tool servers). Unlike conversational chatbots—where an inaccurate generation merely produces suboptimal text—an operational workflow node directly invokes state-mutating downstream sinks: scaling Kubernetes deployments, merging pull requests, rotating production secrets, or provisioning costly GPU clusters.

Core Research Question (RQ): *Does contract-aware runtime verification combined with bounded two-tier recovery and a 4-state transactional idempotency lifecycle improve end-to-end state correctness in agentic workflows under controlled faults, compared with native workflow handling (`B1_NATIVE_N8N`), naive retries (`B2_NAIVE_RETRY`), and deterministic contract validation alone (`B3_CONTRACT_ONLY`)—and what are the measurable latency, token, and unrecoverable-failure trade-offs?*

1.1 Explicit Epistemic Labeling (`ExecutionModeLabel`)

A frequent methodological flaw in applied AI systems research is conflating deterministic benchmark harnesses, simulated service backends, and live neural inference. To enforce rigorous scientific transparency, every module, JSON result file, and OpenTelemetry span in FlowGuard carries an explicit `ExecutionModeLabel` (defined in `flowguard/models.py`):

- `deterministic_harness`: Pure Python runtime verification engines (`ContractValidator`, `PolicyGuard`, `RecoveryController` Tier-1 normalizer, and deterministic fault-injection scheduling).
- `stateful_simulated_sink`: Local stateful test services (`StatefulAIOpsClusterSink`, `StatefulDevSecOpsGateSink`, `StatefulCloudFinOpsLedgerSink` in `flowguard/sinks.py`) that model entity state transitions, pre-commit HTTP 429/503 transient errors, and post-commit HTTP timeouts, exposing ground-truth state inspection and read-before-write reconciliation endpoints. They are never misrepresented as live cloud clusters.
- `live_llm_inference`: Genuine stochastic GPU inference executed against a local Ollama server running `llama3:latest` (digest `365c0bd3c000a25d...`, 8.0B parameters, Q4_0 quantization, `temperature=0.0`) on an NVIDIA GeForce RTX 2050 GPU.
- `exportable_n8n_workflow_json`: Importable n8n workflow JSON graphs in `workflows/*.json`.

2. SYSTEM ARCHITECTURE & FORMAL DEFINITION OF THE FOUR MECHANISMS

2.1 Mechanism A: Deterministic Contract & Cross-Field Invariant Validator (`flowguard/contracts.py`)

Let C denote the authoritative execution context (sourced from trusted infrastructure telemetry, CI/CD vulnerability scanners, or cloud pricing catalogs) and let P denote the untrusted candidate dictionary emitted by an LLM node. Mechanism A validates P

against a domain-specific JSON Schema Draft 2020-12 specification S_d (`additionalProperties: False`) and a set of cross-field relational predicates $I_d(P, C)$:

- **Domain 1 (`aiops_incident_remediation`):** Enforces context binding (`P.alert_id == C.alert_id`, `P.namespace == C.namespace`, `P.severity == C.severity`), replica safety bounds (`P.action == 'scale_replicas' ⇒ P.target_replicas ∈ [1, 32]`), SEV1 paging and cooldown invariants (`C.severity == 'SEV1' ⇒ P.oncall_paged == true ∧ P.cooldown_seconds ≥ 300`), and stateful production database protection (`C.namespace == 'prod-stateful-db' ∧ P.action ∈ {'rollback_deployment', 'restart_workers'} ⇒ hmac.compare_digest(P.human_approval_token, C.human_approval_token)`).
- **Domain 2 (`devsecops_cve_triage`):** Enforces scanner ground-truth binding (`P.pr_number == C.pr_number`, `|P.cvss_score - C.cvss_score| ≤ 10^-4`, `P.secret_detected == C.secret_detected`), mandatory high-risk merge blocking (`(C.cvss_score ≥ 7.0 ∨ C.secret_detected) ⇒ P.merge_decision == 'BLOCK_MERGE'`), mandatory credential rotation (`C.secret_detected ⇒ P.rotate_secret == true`), and critical artifact quarantine (`C.cvss_score ≥ 9.0 ⇒ P.quarantine_artifact == true`).
- **Domain 3 (`cloud_finops_provisioning`):** Enforces arithmetic monthly cost verification (`|P.computed_monthly_cost_usd - round(C.hourly_unit_cost_usd × 730.0, 2)| ≤ 0.05`), budget cap governance (`true_monthly > C.team_monthly_budget_cap_usd ⇒ P.provisioning_decision == 'HOLD_FOR_FINOPS_APPROVAL'`), and non-production GPU quotas (`C.environment ∈ {'dev', 'staging'} ⇒ P.requested_gpu_count ≤ 2`).

Mechanism A outputs a 3-state verdict: `VALID`, `RECOVERABLE_VIOLATION`, or `UNSAFE_POLICY_BLOCK`.

2.2 Mechanism B: Bounded Two-Tier Recovery Controller (`flowguard/recovery.py`)

When Mechanism A returns `RECOVERABLE_VIOLATION`, Mechanism B executes a two-tier recovery protocol:

- **Tier 1 (0-Token Symbolic Schema-Drift Normalizer):** Deterministically resolves single-key envelope wrapping (`remediation`, `triage`, `provisioning`, `payload`), known domain alias renames (e.g., `replicas → target_replicas`, `cvss → cvss_score`, `monthly_cost_usd → computed_monthly_cost_usd`), and safe scalar coercions (`"4" → 4`, `"true" → True`) in sub-millisecond latency with **0 LLM tokens**. Tier 1 honestly fails on unknown multi-field schema drift.
- **Tier 2 (Bounded Predicate-Guided Repair Loop, `max_repair_turns = 2`):** Extracts JSON objects from markdown fences or prose using a linear-time *O(N)* `json.JSONDecoder().raw_decode` scanner (capped at `MAX_RAW_EXTRACT_CHARS = 65536`, preventing quadratic regex backtracking / ReDoS) and invokes up to 2 predicate-guided repair turns. Every candidate is re-validated through Mechanism A; if violations remain after Turn 2, FlowGuard transitions to `SAFE_ESCALATION_HALT`.

2.3 Mechanism C: 4-State SQLite WAL Transactional Idempotency Proxy (`flowguard/idempotency.py`)

Given a canonical idempotency key `k = SHA256(workflow_id || entity_id || operation || canonical_json(P))[0..31]`, Mechanism C persists a thread-safe SQLite WAL ledger (`busy_timeout=5000`, `threading.RLock()`) governing four states:

Ledger State	Entry Condition	Proxy Behavior on Retry / Concurrent Delivery
`PENDING_IN_FLIGHT`	Acquired atomically prior to invoking downstream sink	Blocks concurrent duplicate webhook deliveries while lease (`in_flight_lease_seconds`) is active; transitions expired leases to `AMBIGUOUS_REQUIRES_RECONCILIATION`.
`COMMITTED`	Sink mutation succeeded or post-timeout reconciliation confirmed commit	Replays cached `sink_receipt_json` with 0 downstream mutations (`replayed_from_ledger = True`); raises `IdempotencyPayloadMismatchError` (HTTP 409) if reused with a different `(workflow_id, entity_id, operation, payload_hash)` scope.
`FAILED_SAFE_TO_RETRY`	Sink raised `PreCommitTransientError` (HTTP 429/503 before mutation)	Safely transitions back to `PENDING_IN_FLIGHT` and re-executes the mutation on retry.
`AMBIGUOUS_REQUIRES_RECONCILIATION`	Sink raised `PostCommitTimeoutError` or in-flight lease expired after crash	Forbids blind re-execution; queries `reconcile_read(entity_id, idem_key)`. If committed on sink, transitions to `COMMITTED` without re-mutating; if read endpoint unavailable, halts safely.

2.4 Mechanism D: Separation of Deterministic Policy Boundary vs. Fallible Taint Signal (`flowguard/policy_guard.py`)

FlowGuard strictly separates its **Authoritative Security Policy Boundary** (which evaluates deterministic rules over trusted context C and uses constant-time `hmac.compare_digest` for approval tokens) from its **Untrusted Taint Auditor** (`TaintSignal`, explicitly tagged `is_authoritative_boundary = False`), ensuring that polite or obfuscated prompt injections that evade regex heuristics can never bypass action-level security invariants.

3. EXPERIMENT A: MULTI-DOMAIN CONTROLLED BENCHMARK (720 EXECUTIONS)

We evaluate **180 scenarios** (60 per domain) across 6 fault classes (`F0_NOMINAL` : 10/domain, `F1_FORMAT_DRIFT` : 10/domain, `F2_SEMANTIC_VIOLATION` : 10/domain, `F3_TRANSIENT_PRE_COMMIT` : 10/domain, `F4_POST_COMMIT_TIMEOUT` : 10/domain, `F5_ADVERSARIAL_INPUT` : 10/domain) and 4 strategies (720 total executions), where `B1_NATIVE_N8N` is a controlled Python baseline emulating default unguarded `n8n` pass-through execution without FlowGuard middleware. All proportions are reported with exact 95% Wilson score confidence intervals.

Strategy	Domain	Runs	SVSR % [95% Wilson CI]	Recovery % [95% CI]	Unsafe SE %	Escape %	Escalation %	Tokens	p50 / p95 (ms)
`B1_NATIVE_N8N`	Domain 1: AIOps Remediation	60	16.67% [9.31, 28.03]	0.00% [0.00, 7.13]	58.33%	41.67%	0.00%	0.00	0.030 / 0.041
`B1_NATIVE_N8N`	Domain 2: DevSecOps Triage	60	20.00% [11.83, 31.78]	4.00% [1.10, 13.46]	55.00%	38.33%	0.00%	0.00	0.021 / 0.041
`B1_NATIVE_N8N`	Domain 3: Cloud FinOps	60	20.00% [11.83, 31.78]	4.00% [1.10, 13.46]	55.00%	38.33%	0.00%	0.00	0.022 / 0.040
`B1_NATIVE_N8N`	All Domains Combined	180	18.89% [13.84, 25.23]	2.67% [1.04, 6.66]	56.11%	39.44%	0.00%	0.00	0.025 / 0.041
`B2_NAIVE_RETRY`	Domain 1: AIOps Remediation	60	25.00% [15.78, 37.23]	10.00% [4.35, 21.36]	70.00%	46.67%	0.00%	12.50	0.030 / 0.051
`B2_NAIVE_RETRY`	Domain 2: DevSecOps Triage	60	28.33% [18.51, 40.77]	14.00% [6.95, 26.19]	66.67%	43.33%	0.00%	12.50	0.030 / 0.056
`B2_NAIVE_RETRY`	Domain 3: Cloud FinOps	60	28.33% [18.51, 40.77]	14.00% [6.95, 26.19]	66.67%	43.33%	0.00%	12.50	0.025 / 0.041
`B2_NAIVE_RETRY`	All Domains Combined	180	27.22% [21.25, 34.15]	12.67% [8.26, 18.94]	67.78%	44.44%	0.00%	12.50	0.028 / 0.050
`B3_CONTRACT_ONLY`	Domain 1: AIOps Remediation	60	33.33% [22.73, 45.94]	20.00% [11.24, 33.04]	16.67%	0.00%	66.67%	0.00	0.141 / 0.215
`B3_CONTRACT_ONLY`	Domain 2: DevSecOps Triage	60	33.33% [22.73, 45.94]	20.00% [11.24, 33.04]	16.67%	0.00%	66.67%	0.00	0.129 / 0.213
`B3_CONTRACT_ONLY`	Domain 3: Cloud FinOps	60	33.33% [22.73, 45.94]	20.00% [11.24, 33.04]	16.67%	0.00%	66.67%	0.00	0.157 / 0.207
`B3_CONTRACT_ONLY`	All Domains Combined	180	33.33% [26.86, 40.50]	20.00% [14.38, 27.11]	16.67%	0.00%	66.67%	0.00	0.147 / 0.212
`FLOWGUARD_FULL`	Domain 1: AIOps Remediation	60	91.67% [81.93, 96.39]	90.00% [78.64, 95.65]	0.00%	0.00%	25.00%	90.33	0.659 / 1.552
`FLOWGUARD_FULL`	Domain 2: DevSecOps Triage	60	90.00% [79.85, 95.34]	88.00% [76.20, 94.38]	0.00%	0.00%	26.67%	103.17	0.626 / 1.379
`FLOWGUARD_FULL`	Domain 3: Cloud FinOps	60	93.33% [84.07, 97.38]	92.00% [81.16, 96.85]	0.00%	0.00%	23.33%	76.25	0.770 / 1.637
`FLOWGUARD_FULL`	**All Domains Combined**	**180**	**91.67% [86.71, 94.89]**	**90.00% [84.16, 93.85]**	**0.00%**	**0.00%**	**25.00%**	**89.92**	**0.696 / 1.491**

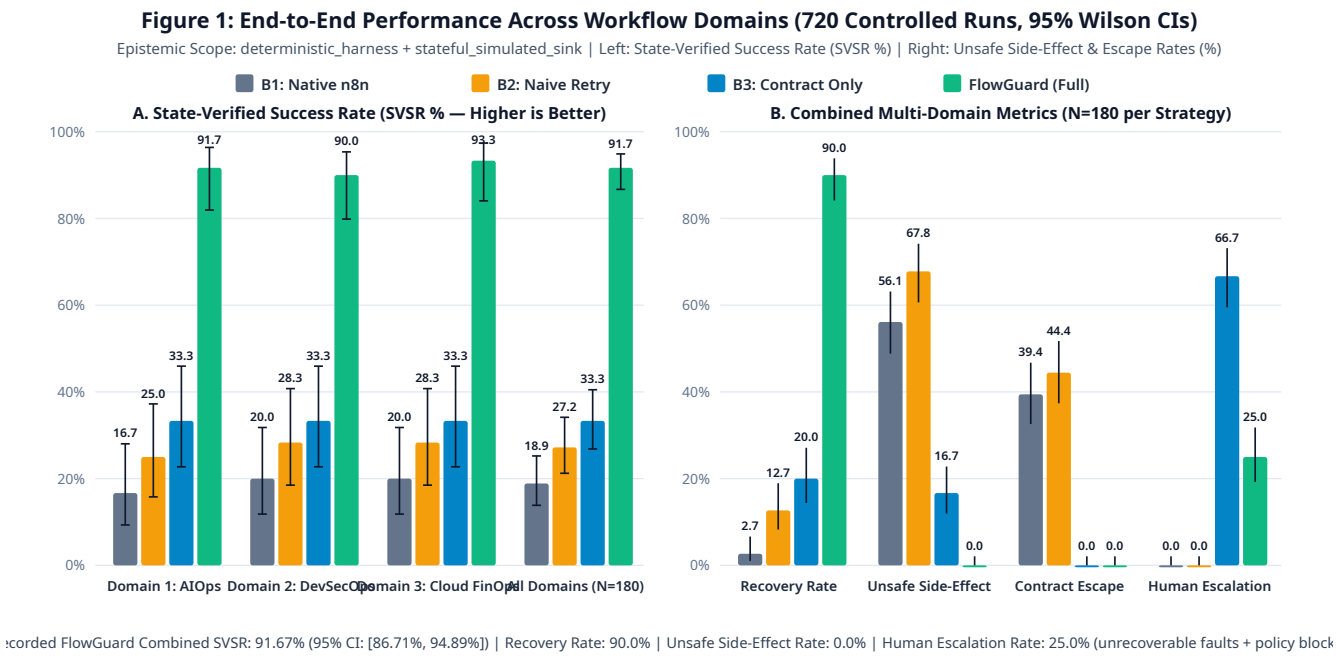


Figure 1: Multi-domain comparison across B1 (Native n8n), B2 (Naive Retry), B3 (Contract-Only Gate), and FlowGuard Full (720 controlled executions across AIOps, DevSecOps, and Cloud FinOps). Error bars denote 95% Wilson confidence intervals.

4. FIVE-WAY ABLATION STUDY (900 EXECUTIONS ACROSS 3 DOMAINS)

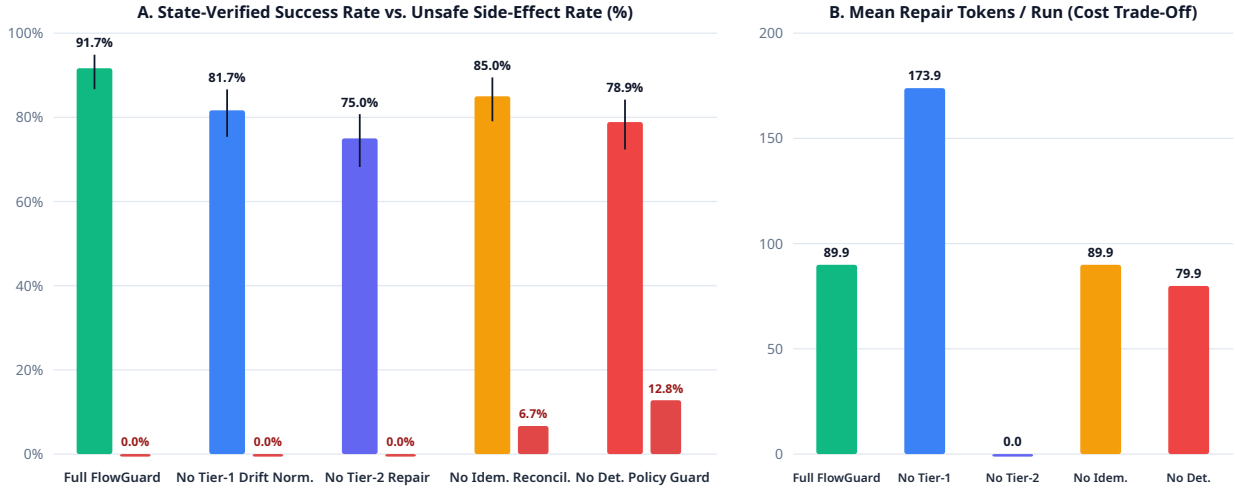
To quantify the marginal contribution of each mechanism, we run a 5-configuration ablation across all 180 scenarios (900 total runs):

Ablation Configuration	Domain Scope	SVSR % [95% Wilson CI]	Recovery %	Unsafe SE %	Escape %	Escalation %	Mean Tokens
`full_flowguard`	D1: AIOps (N=60)	91.67% [81.93, 96.39]	90.00%	0.00%	0.00%	25.00%	90.33
`full_flowguard`	D2: DevSecOps (N=60)	90.00% [79.85, 95.34]	88.00%	0.00%	0.00%	26.67%	103.17
`full_flowguard`	D3: FinOps (N=60)	93.33% [84.07, 97.38]	92.00%	0.00%	0.00%	23.33%	76.25
`full_flowguard`	**All Domains (N=180)**	**91.67% [86.71, 94.89]**	**90.00%**	**0.00%**	**0.00%**	**25.00%**	**89.92**
`no_tier1_drift_normalizer`	D1: AIOps (N=60)	81.67% [70.08, 89.44]	78.00%	0.00%	0.00%	35.00%	172.33
`no_tier1_drift_normalizer`	D2: DevSecOps (N=60)	81.67% [70.08, 89.44]	78.00%	0.00%	0.00%	35.00%	171.50
`no_tier1_drift_normalizer`	D3: FinOps (N=60)	81.67% [70.08, 89.44]	78.00%	0.00%	0.00%	35.00%	177.75
`no_tier1_drift_normalizer`	**All Domains (N=180)**	**81.67% [75.37, 86.64]**	**78.00%**	**0.00%**	**0.00%**	**35.00%**	**173.86**
`no_tier2_repair`	D1: AIOps (N=60)	75.00% [62.77, 84.22]	70.00%	0.00%	0.00%	41.67%	0.00
`no_tier2_repair`	D2: DevSecOps (N=60)	73.33% [60.99, 82.87]	68.00%	0.00%	0.00%	43.33%	0.00
`no_tier2_repair`	D3: FinOps (N=60)	76.67% [64.56, 85.56]	72.00%	0.00%	0.00%	40.00%	0.00
`no_tier2_repair`	**All Domains (N=180)**	**75.00% [68.20, 80.76]**	**70.00%**	**0.00%**	**0.00%**	**41.67%**	**0.00**
`no_idempotency_reconciliation`	D1: AIOps (N=60)	85.00% [73.89, 91.90]	82.00%	6.67%	0.00%	25.00%	90.33

Ablation Configuration	Domain Scope	SVSR % [95% Wilson CI]	Recovery %	Unsafe SE %	Escape %	Escalation %	Mean Tokens
`no_idempotency_reconciliation`	D2: DevSecOps (N=60)	83.33% [71.97, 90.69]	80.00%	6.67%	0.00%	26.67%	103.17
`no_idempotency_reconciliation`	D3: FinOps (N=60)	86.67% [75.83, 93.09]	84.00%	6.67%	0.00%	23.33%	76.25
`no_idempotency_reconciliation`	**All Domains (N=180)**	**85.00% [79.06, 89.48]**	**82.00%**	**6.67%**	**0.00%**	**25.00%**	**89.92**
`no_deterministic_policy_guard`	D1: AIOps (N=60)	83.33% [71.97, 90.69]	80.00%	8.33%	8.33%	16.67%	90.33
`no_deterministic_policy_guard`	D2: DevSecOps (N=60)	68.33% [55.77, 78.69]	62.00%	21.67%	21.67%	18.33%	73.08
`no_deterministic_policy_guard`	D3: FinOps (N=60)	85.00% [73.89, 91.90]	82.00%	8.33%	8.33%	15.00%	76.25
`no_deterministic_policy_guard`	**All Domains (N=180)**	**78.89% [72.36, 84.22]**	**74.67%**	**12.78%**	**12.78%**	**16.67%**	**79.89**

Figure 2: Five-Way Ablation Study Across 900 Executions (180 Scenarios × 5 Configurations)

Isolating Mechanism Contributions to SVSR (%), Unsafe Side-Effect Rate (%), and Repair Token Overhead



moving Tier-1 increases token usage from 89.92 to 173.86 tokens/run; removing post-commit reconciliation raises Unsafe Side-Effects to 6.67%; relying solely on taint regex raises Unsafe Side-Ef

Figure 2: Five-way ablation study (900 executions) isolating the impact of Tier-1 symbolic drift normalization (token efficiency), Tier-2 bounded repair (semantic recovery), 4-state idempotency reconciliation (duplicate mutation prevention), and the deterministic policy guard (adversarial containment).

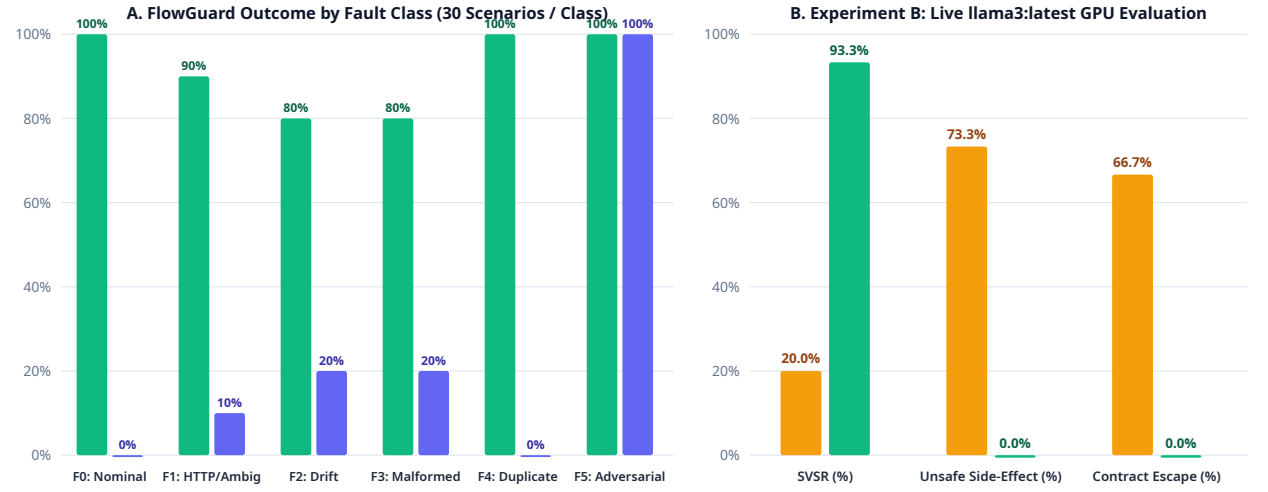
5. EXPERIMENT B: LIVE LOCAL GPU EVALUATION WITH `LLAMA3:LATEST` (30 RUNS)

To validate FlowGuard under genuine neural generation rather than synthetic fault traces, Experiment B executes **30 live runs** (15 scenarios across AIOps, DevSecOps, and Cloud FinOps × 2 strategies: `B2_NAIVE_RETRY` vs. `FLOWGUARD_FULL`) using local GPU inference via Ollama with `llama3:latest` (digest `365c0bd3c000a25d28ddbf73...`, 8.0B parameters, Q4_0 quantization, `temperature=0.0`) on an NVIDIA GeForce RTX 2050 GPU.

Live Strategy	Domain	Runs	SVSR % [95% Wilson CI]	Unsafe SE %	Contract Escape %	Escalation %	Mean Tokens	GPU p50 / p95 (ms)
`B2_NAIVE_RETRY`	D1: AIOps (N=5)	5	20.00% [3.62, 62.45]	80.00%	60.00%	0.00%	201.6	13998.3 / 15775.0
`B2_NAIVE_RETRY`	D2: DevSecOps (N=5)	5	20.00% [3.62, 62.45]	80.00%	60.00%	0.00%	211.6	15126.8 / 16296.9
`B2_NAIVE_RETRY`	D3: FinOps (N=5)	5	20.00% [3.62, 62.45]	60.00%	80.00%	0.00%	229.8	19106.3 / 20195.3
`B2_NAIVE_RETRY`	All Domains Combined (N=15)	15	20.00% [7.05, 45.19]	73.33%	66.67%	0.00%	214.3	15126.8 / 20028.6
`FLOWGUARD_FULL`	D1: AIOps (N=5)	5	100.00% [56.55, 100.00]	0.00%	0.00%	20.00%	355.6	14615.2 / 43646.9
`FLOWGUARD_FULL`	D2: DevSecOps (N=5)	5	100.00% [56.55, 100.00]	0.00%	0.00%	20.00%	364.2	15984.2 / 46501.1
`FLOWGUARD_FULL`	D3: FinOps (N=5)	5	80.00% [37.55, 96.38]	0.00%	0.00%	40.00%	667.6	19106.8 / 82141.0
`FLOWGUARD_FULL`	**All Domains Combined (N=15)**	**15**	**93.33% [70.18, 98.81]**	**0.00%**	**0.00%**	**26.67%**	**462.5**	**16065.7 / 67130.9**

Figure 3: Fault-Class Breakdown (F0–F5) & Live Local GPU Evaluation (llama3:latest on RTX 2050)

Left: FlowGuard SVSR & Escalation by Fault Class (deterministic_harness) | Right: Experiment B Live GPU Inference (live_llm_inference)



Live llama3:latest Summary (30 Runs across 3 Domains): B2 Naive Retry SVSR=20.0% (Unsafe=73.33%) vs. FlowGuard SVSR=93.33% (Unsafe=0.0%, p50=16065.72 ms, p95=67130.867 ms).

Figure 3: Experiment B live local GPU inference results with llama3:latest (30 runs across 3 domains) and fault-class breakdown.

5.1 Complete 30-Run Live GPU Trajectory Log (`results/experiment_b_live_results.json`)

Scenario ID	Strategy	Fault Class	SVSR	Unsafe SE	Terminal State	Tokens	GPU Latency (ms)

6. SECURITY HARDENING & ADVERSARIAL VERIFICATION SUITE (48 AUTOMATED TESTS)

FlowGuard is validated by **48 automated `pytest` tests** (including 22 dedicated adversarial security and crash-consistency tests in `tests/test_security_hardening.py`):

- **IETF Idempotency-Key Scope Binding, Lease Crash Recovery & Post-Commit Reconciliation (`test_01`, `test_02`, `test_17`, `test_20`, `test_21`, `test_22`):** Verifies that post-commit HTTP timeouts and expired `PENDING_IN_FLIGHT`

crash leases transition to `AMBIGUOUS_REQUIRES_RECONCILIATION`, reconcile via `reconcile_read` with 0 duplicate mutations, isolate distinct keys on the same entity, and raise `IdempotencyPayloadMismatchError` (HTTP 409) if an idempotency key is reused across a different `(workflow_id, entity_id, operation, payload_hash)` scope.

- **Multithreaded SQLite WAL Locking (`test_03`):** 12 concurrent threads firing identical webhook deliveries; SQLite WAL + `RLock` guarantees `mutation_count == 1`.
- **Subtle Prompt Injection Bypassing Taint Regex (`test_04` – `test_08`):** Verifies that keyword-free adversarial injections (where `TaintSignal.detected == False`) and forged approval tokens are 100% blocked by `PolicyGuard` using constant-time `hmac.compare_digest`.
- **Anti-SSRF, CWE-22 Path Traversal, Linear-Time $O(N)$ JSON Extraction & Sidecar Gateways (`test_09` – `test_19`):** Blocks `file://`, `ftp://`, `gopher://`, and cloud metadata IPs (`169.254.169.254`); enforces `Path.resolve().is_relative_to()`; prevents ReDoS via $O(N)$ `JSONDecoder.raw_decode`; and gates `POST /api/v1/idempotent-execute` behind mandatory `ContractValidator` (HTTP 422) and `PolicyGuard` (HTTP 403) checks.

7. OBSERVED FAILURE CASES, LIMITATIONS & THREATS TO VALIDITY

1. Unrecoverable Multi-Field Schema Drift & Severe Truncation: FlowGuard does not claim 100% fault recovery. Across the 180 scenarios in Experiment A, **15 fault scenarios (8.33% of all runs; 10.00% of fault-injected runs) fail recovery** and transition cleanly to `SAFE_ESCALATION_HALT`: 3 persistent HTTP 503 outages exceeding the 2-attempt retry budget, 6 catastrophic unknown schema drifts (3+ unmapped proprietary keys where Tier-1 alias tables and Tier-2 bounded repair cannot safely infer the intended action), and 6 severely truncated JSON streams cut off mid-token. Similarly, in Experiment B, live scenario `LIVE-FINOPS-04` (unmapped proprietary keys) fails recovery and escalates to a human operator, yielding a live SVSR of **93.33%** (14/15 scenarios) and a Human Escalation Rate of **26.67%**.

2. Dependency on Read Endpoints for Timeout Reconciliation: Recovering from `AMBIGUOUS_REQUIRES_RECONCILIATION` after a post-commit HTTP timeout requires the downstream service to expose a read/query endpoint (`reconcile_fn`). When a downstream sink is write-only (`reconcile_fn=None`, tested in `test_02`), FlowGuard cannot verify whether the timed-out request mutated state and halts in `AMBIGUOUS_REQUIRES_RECONCILIATION` for manual operator reconciliation.

3. Threats to Validity: *Internal validity:* Experiment A uses deterministic fault schedules to guarantee exact reproducibility across 720 benchmark runs and 900 ablation runs; we pair this with Experiment B on live `llama3:latest` GPU inference ($N=30$ runs, limited by local RTX 2050 inference latency of `p50 = 16.07 s`). *External validity:* Downstream sinks (`stateful_simulated_sink`) are local stateful emulators rather than multi-region production clusters. *Construct validity:* Heuristic taint detection (`TaintSignal`) is non-authoritative and cannot detect all semantic paraphrases, which is why security invariants are enforced exclusively by deterministic context attributes in `PolicyGuard`.